

GALACTIC COLONIZATION:

DEEP SPACE EXPLORER



 FUEL ■■■■■■■	 COLONISTS ■■■■■■■	 COLONIES ■■■■■■■	 SHIELDS ■■■■■■■	 THREATS ■■■■■■■
---	---	--	--	---

EXPLORE. SURVIVE. COLONIZE.

Technical Project Report

ARTIFICIAL INTELLIGENCE ALGORITHMS

Marco Oliveri

Galactic Colonization: Deep Space Explorer
*Final technical project report for the Artificial Intelligence
Algorithms course.*

1. Project Objective

Galactic Colonization: Deep Space Explorer is a space exploration simulator in which a spaceship travels through a procedurally generated universe, gathers resources, and faces random events tied to the danger level of the visited sectors.

The assignment required modeling the universe as a graph, ensuring sector reachability, managing fuel and resources, recording discoveries in the galactic catalog, and automating movement decisions using an autonomous driving algorithm.

2. General Architecture

The architecture follows a tiered division: custom data structures provide the implementation foundations, Game Models represent the domain state, Game Logic coordinates the behavior, and the graphical interface visualizes the simulation without duplicating the core logic.

Element	Description
Sectors	Nodes of the universe with an ID, danger level, resources, and visit status.
Routes	Hyperspace links weighted by fuel cost.

Element	Description
Spaceship	Entity maintaining the current position, fuel, and gathered resources.
GameEngine	Coordinator of the game flow, events, autopilot, catalog, and turns.
Interface	Python/Pygame graphical visualization of the automatic simulation.

Module	Main Responsibility	Classes/Files
Custom Data Structures	Lists, hash table, graph, priority queue, and disjoint set.	DataStructures.py
Game Models	Sectors, routes, spaceship, and procedural universe.	GameModels.py
Game Logic	Turns, events, catalog, autopilot, and simulation.	GameEngine.py, AutoPilotAI.py, GalacticCatalog.py
User Interface	Cover, soundtrack, graph, logs, and final summary.	ui/ui_game.py, ui/resources/

3. Data Structures and Models

Main Data Structures (Exported)

- **Graph:** Imported into `GameModels.py`. It forms the core of the `UniverseGraph` class (in the `self.core_graph` variable) to map galactic sectors and hyperspace routes.
- **LinkedList:** Imported into `GameModels.py` to manage groups of nodes, such as the global list of sectors or the adjacent and yet-to-be-visited nodes. It is also used in `GalacticCatalog.py` to historically store the routes discovered by the spaceship (`self.known_routes`).
- **HashTable:** Used in `AutoPilotAI.py` to keep track of currently visited nodes during the recursive exploration of the backtracking algorithm. It is also imported into `GalacticCatalog.py` to manage the inventory of visited sectors (`self.visited_sectors`).
- **MinPriorityQueue:** Employed in `GameModels.py` within the `generate_procedural_universe` function. Its task is to sort the edges based on fuel cost to process them in ascending order.

- **DisjointSet**: Also used in `GameModels.py` during the procedural generation of the universe. It works in synergy with the `MinPriorityQueue` (forming Kruskal’s algorithm) to connect sectors while ensuring no unwanted cycles are created in the initial structure.

Supporting Classes (Internal)

- **Node, HeapNode, Array, HashEntry, Edge**: These classes are not imported or instantiated in external modules. However, they are constantly utilized within `DataStructures.py` as fundamental components. For instance, `Array` forms the buckets of the `HashTable` and the memory space of the `MinPriorityQueue`, while `Edge` mathematically defines the connections within the `Graph`.

4. Procedural Universe Generation

The generation randomly creates sectors with random danger levels and resources. The first sector is the launch point and is initialized with no danger or resources, so the simulation starts from a controlled state.

To ensure reachability, Kruskal’s algorithm was used with a custom `Disjoint Set` and `MinPriorityQueue`. Possible routes are generated with a random cost, then selected avoiding cycles in the initial phase. Afterwards, extra edges are added to create loops and make the topology more interesting.

- Connectivity validated via `is_connected()`.
- Each sector respects the maximum limit of five connections.
- `ensure_connectivity()` checks and completes any missing connections.
- `enforce_max_connections()` reports violations of the maximum connection limit.

5. Game Logic and Autopilot

The `GameEngine` initializes the universe, spaceship, catalog, and autopilot; it manages the arrival in sectors, resource extraction, and unexpected events. The simulation ends when fuel runs out or no more sustainable routes exist.

The Autopilot is the algorithm that automatically guides the spaceship through the game universe, choosing at each step the next sector that maximizes the resources gathered along the path, taking into account the remaining fuel and the danger level of each sector.

Data Structures Used

- **Weighted graph (Graph)**: represents the universe, where nodes are sectors (**Sector**) and edges are hyperspace routes (**Edge**) with a weight equal to the fuel cost. The adjacency list is implemented using a **HashTable**.
 - **HashTable**: used in the Autopilot as the set of visited nodes, to avoid cycles during recursive exploration.
-

Algorithm — DFS with Backtracking

1. DFS Root Level: Choosing the First Step

The `calculate_next_move(current, fuel_left)` method represents the **root level** of the DFS exploration tree. It iterates over all direct neighbors of the current sector (traversing the adjacency list) and, for each reachable neighbor (whose fuel cost does not exceed the remaining fuel), calculates the heuristic, a **risk-weighted resource score**:

$$\text{weighted_resources} = \text{resources} \times \left(1 - \frac{\text{danger_level}}{100}\right)$$

This factor penalizes high-danger sectors, favoring safer ones given equal resources.

2. Recursive Depth-First Exploration (DFS + Backtracking)

For each candidate neighbor, the `_backtracking(neighbor, fuel, visited)` method is invoked, which performs a **DFS (Depth-First Search) with backtracking**, recursively exploring in-depth all future paths starting from that sector. Unlike a standard DFS (which permanently marks nodes as visited), backtracking **restores the state** after each exploration, allowing the enumeration of all possible paths. In detail:

- **Explores** every outgoing edge from the current sector.
- **Verifies** two feasibility conditions: (a) the edge cost does not exceed the remaining fuel, (b) the destination node has not already been visited.
- **Marks** the node as visited in the **HashTable** before exploring the subtree.
- **Restores** the state (backtrack) by resetting the flag to **False** after exploring the subtree, allowing the exploration of alternative paths that pass through the same node.
- **Accumulates** the risk-weighted resources along each path and returns the maximum value found.

- **Terminates** (base case) when fuel runs out or no feasible edges exist.

3. Selection of the Optimal Move

The adjacent sector that produces the **maximum cumulative score** (its own weighted resources + the best future exploration via backtracking) is selected as the next move.

6. Galactic Catalog and Events

The GalacticCatalog records visited sectors, traveled routes, and extracted resources. Unexpected occurrences are modeled as different events, with different names and effects on fuel and resources.

Event	Effect
Solar Storm	Fuel reduction due to shield damage.
Space Pirates	Fuel loss and possible resource loss.
Gravitational Anomaly	High fuel consumption to correct the route.
Asteroid Field	Evasive maneuvers with possible resource loss.

7. Graphical Interface

The graphical interface was built in Python using Pygame. The user observes an autonomous simulation: upon startup, a cover with a looping soundtrack appears; by pressing Start (Avvia), the graph is displayed and the spaceship proceeds automatically.

- Initial screen with a cover image and a Start button.
- Looping soundtrack until the application is closed.
- Visual graph showing visited and unvisited sectors, the ship's position, and the last highlighted route.
- Side panel displaying fuel, resources, simulation status, turns, and available routes.
- Separate event log and an automatic final summary showing the path, resources, fuel, and routes.